Ders 22 Genişletilmiş Notu: Üretici-Tüketici Problemi, Swing ile GUI ve Olay Yönetimi, Katmanlı Mimari, MVC, MVP ve MVVM

Ders 22 Genişletilmiş Notu: Üretici-Tüketici Problemi, Swing ile GUI ve Olay Yönetimi, Katmanlı Mimari, MVC, MVP ve MVVM

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Üretici-Tüketici Problemi
- 2. Java Swing ile Pencere Oluşturma
- 3. Butona Görev Atamak: İlk Olay Dinleyicisi
- 4. Aradan Sonra: Üretici-Tüketici Koduna Dönüş
- 5. Olay Yönetiminin Ayrıntıları
- 6. Temel Bileşenler ve Python'daki Karşılığı
- 7. Katmanlı Mimari
- 8. MVC (Model-View-Controller)
- 9. MVP (Model-View-Presenter)
- 10. MVVM (Model-View-ViewModel)
- 11. Android Üzerinde Giriş Ekranı Örneği
- 12. Clean Architecture
- 13. Veri Tabanı Sorguları, ORM ve Yazılım Tasarımı

1. Üretici-Tüketici Problemi

Hoca, dersin notlarının geçen hafta thread konusuyla bittiğini söyledi. Bu derste ek olarak bir örnek yapmak istediğini, bu örneğin gelecek dönem işletim sistemleri dersinde de işe yaracağını belirtti. Örnek, üretici-tüketici (producer-consumer) problemidir.

1.1 Problemin Tanımı

Üretimin ve tüketimin yapıldığı bir kaynak düşünelim. Bu kaynak bir dizi gibi düşünülebilir, ama başka bir yapı da olabilir. Kaynağa eleman ekleyen bir mekanizma (push, insert, add) ve kaynaktan eleman çıkaran bir mekanizma (pop, delete, remove) vardır. Kaynağın bir kapasitesi vardır. Ürünler sırayla 0, 1, 2, … diye eklenir.

Bu yapıda iki kural vardır:

- **Kaynak boşsa tüketici bekler.** Bir şey almak için kaynakta alınacak bir şey olmalıdır. Pop yapan işlem çok hızlı çalışıp kaynağı boşalttıysa bu thread beklemeye geçer, push yapan thread'in çalışması gerekir.
- **Kaynak (buffer) doluysa üretici bekler.** Push yapan thread kaynağı doldurduysa beklemeye geçer, tüketicinin tüketmesi gerekir.

Ayrıca elemanların sırası korunmalıdır: Önce 0. eleman, sonra 1. eleman, sonra 2. eleman alınır. Programda üç thread vardır: üretici, tüketici ve main. Bütün bu işlerin senkron yapılması gerekir. Veri yapısı olarak `LinkedList` bir kuyruk (queue) mantığıyla kullanılacaktır.

1.2 Sınıfın İlk Hali

Hoca bir `PC` (producer-consumer) sınıfı açtı. Sınıfta `Integer` türünde bir `LinkedList`, bir kapasite (`capacity`) değeri ve üretim sayısını tutan bir sayaç vardır. Listeyi doğrudan alanda ilklendirdiği için kurucu metoda (constructor) gerek olmadığını söyledi.

Sınıfta thread'lerle çalıştırılacak iki metot olacaktır: `produce` ve `consume`. Sayaç (`number`) `static` tanımlandı ve 0 ile başlatıldı. Hocanın anlatımıyla `produce` metodu şöyle işler:

1. `value` değişkeni 0 ile başlar. Bu, üretilecek değerdir.
2. Sayaç 10 olana kadar dönen bir `while` döngüsü vardır.
3. Listenin boyu kapasiteye eşitse thread beklemelidir. Bunun için bir boş döngü yazılabilir, ama Java'nın hazır `wait()` metodu vardır. `wait()` için derleyici `InterruptedException` uyarısı verir. Hoca metoda `throws InterruptedException` ekledi.
4. Beklemeye düşülmediyse ekrana "üretici üretti" mesajı ve üretilen değer yazılır.
5. Sayaç bir artırılır, `value` listeye eklenir, sonra `value` bir artırılır. `add` elemanı listenin sonuna ekler.
6. `notify()` ile diğer thread uyarılır: Benim işim bitti, kaynaktan çıkıyorum.
7. Çalışmayı gözle izleyebilmek için `Thread.sleep(500)` ile bekleme verilir.

Hoca bu metodu bir işçiye benzetti: İşçi sabah mesaiye başlar, elindeki işi bitirince paydos eder. Metot da bir kere çağrılır, işini bitirince sonlanır, tekrar tekrar çağrılmaz.

`consume` metodu da benzerdir. Yine sayacın 10'dan küçük olduğu sürece dönen bir döngü vardır. Hoca burada sayacı azaltmadığını özellikle söyledi: Sayaç o gün kaç tane üretildiğini tutar, tüketici kuyruktan çıkarır ama sayaca dokunmaz. Liste boşsa (`list.size() == 0`) tüketilecek bir şey yoktur, thread `wait()` ile bekler. Değilse liste kuyruk gibi kullanılır: Ekleme sondan yapıldığı için alma baştan yapılır, bunun için `removeFirst()` vardır. Alınan değer yazdırılır, `notify()` çağrılır, `Thread.sleep(500)` ile beklenir.

> **Ders sonrası not:** Hoca sayacı "kaldığı yerden devam etsin diye" `static` yaptığını söyledi. Bu programda tek bir `PC` nesnesi vardır ve metotlar bir kere çağrılır. Sayacın değeri nesne alanı olarak da korunur: Aşağıdaki Bölüm 4.3'te sayaçları `static` olmayan sürüm javac ile derlenip çalıştırıldı, 10 eleman üretildi ve 10'u da tüketildi.

1.3 Test Sınıfı ve Thread'ler

Test sınıfında önce bir `PC` nesnesi oluşturuldu. İki thread bu nesne üzerinden çalışacaktır. Hoca `Thread` sınıfından kalıtım almak yerine `Thread` nesnesi oluşturdu ve içine `Runnable` arayüzünden anonim bir iç sınıf (inner class) verdi. `Runnable` arayüzünün soyut metodu `run` yazılmak zorundadır. `t1`'in `run` metodu `pc.produce()`, `t2`'nin `run` metodu `pc.consume()` çağırır. `produce` içinde `InterruptedException` oluşabildiği için çağrı `try-catch` içine alındı, `catch` içinde `e.printStackTrace()` yazıldı. `run` metoduna `throws` eklemek yerine `try-catch` yazmanın en doğrusu olduğunu söyledi.

Bu arada `pc.produce` için "cannot find symbol" hatası çıktı, metot görünmüyordu. Bir öğrenci diğer `PC.java` dosyasının kaydedilmesini önerdi. Dosyalar düzenlenince sorun kalktı.

Thread'ler `t1.start()` ve `t2.start()` ile başlatıldı. Ardından `t1.join()` ve `t2.join()` yazıldı. Bu çağrılar için de `InterruptedException` istendi. Hoca önce `try-catch` düşündü, sonra vazgeçip `main` metoduna `throws` ekledi.

Hoca `join` için şunu söyledi: `t1` işini bitirecek, `t2`'yi bekleyeceğiz, sonra `t2` de işini bitirecek. Yani `t2`, `t1`'i, main de `t2`'yi bekliyor.

> **Ders sonrası not:** Hoca "`t2`, `t1`'i bekliyor" dedi; doğrusu, iki `join` çağrısını da main thread yapar ve yalnız main bekler. `t2` ile `t1` birbirini beklemez. Bu, küçük bir programla javac ile denendi: `t1` önce bir süre uyuyor, `t2` uyumuyor. Çıktı sırası "t2 bitti", "t1 bitti", "main devam ediyor" oldu. Yani `join` sırası thread'lerin bitiş sırasını belirlemez, yalnız main'in hangi sırayla bekleyeceğini belirler.

1.4 İlk Çalıştırmalar ve Senkronizasyon

İlk çalıştırmada üretici durmadan üretti, tüketiciye sıra gelmedi. Hoca kodu izleyince bazı satırların `while` döngüsünün dışında kaldığını gördü ve onları döngünün içine aldı.

Yeniden çalıştırınca üretici bir kez üretti, sonra `notify` ile ilgili bir hata çıktı ve thread kapandı. Hoca hatanın "monitör" ile ilgili olduğunu okudu ve nedeni şöyle açıkladı: Paylaşılan bir kaynak (linked list) vardır, buna erişim senkron olmalıdır. Birisi eriştiğinde diğerinin o anda erişmesi engellenmelidir. Aksi halde üretici ilk elemanı daha eklemeyi bitirmeden tüketici oraya erişebilir.

> **Hocanın vurgusu:** Paylaşılan kaynağa erişim senkron olmalıdır. Biri erişirken diğeri erişmemelidir.

Çözüm `synchronized` anahtar kelimesidir. Hoca `while` döngüsünün içinde `synchronized (this)` bloğu açtı. `this`, bu sınıfın o anki nesnesidir. Üretici bloğa girince tüketici aynı anda giremez. Aynı blok `consume` içinde de açıldı. `Thread.sleep` satırı bloğun dışında kaldı.

Bu hatanın nedeni javac ile denendi: `wait()` ve `notify()` `synchronized` dışında çağrıldığında iki thread de `IllegalMonitorStateException: current thread is not owner` hatası verdi.

Bundan sonra program ilerledi. Çıktıda üretici 0, 1, 2, 3'ü üretti, sonra sıra diğer thread'e geçti ve tüketici 0'ı tüketti. Hoca, 0 tüketilmeden 1'in tüketilmesinin söz konusu olmadığını, kuyruk mantığının burada göründüğünü söyledi. Çıktıda üretim ile tüketim sayılarının farklı görünmesi sorun değildir, çünkü yazan değer kaçıncı elemanın tüketildiğidir.

Hoca çift taraflı bir kuyrukta gerçek hayatta aynı anda iki uca erişimin mümkün olabileceğini, ama nesne üzerindeki senkronizasyonun ancak bu şekilde sağlanacağını da ekledi.

> **Ek bilgi (derste anlatılmadı):** `wait()` çağıran thread, tuttuğu kilidi bırakıp bekler. Bu sayede tüketici `wait()` içindeyken üretici aynı `synchronized (this)` bloğuna girip eleman ekleyebilir; bu durum Bölüm 4.3'teki programın üreticisi yavaşlatılarak javac ile denendi. `Thread.sleep` ise büyük olasılıkla tutulan kilidi bırakmaz, bu yüzden derste bloğun dışında kalması uygundur. Birden çok üretici ve tüketici olursa büyük olasılıkla `notifyAll` gerekir, çünkü `notify` yalnız bir thread uyandırır.

1.5 Noktalı Virgül Hatası ve Sayaç Karışıklığı

Tüketim bir yerde tuhaf bir sayıya atlayınca hoca kodu yeniden okudu ve `while (list.size() == 0);` satırının sonunda noktalı virgül olduğunu gördü. Noktalı virgül boş bir döngü gövdesi demektir, alt satırdaki `wait()` döngünün parçası olmaz.

Bu hata javac ile denendi: Tüketici liste boşken bu satıra gelirse kilidi tutarak dönmeye devam eder, üretici bloğa giremez ve program takılır. Liste boş değilken gelirse `wait()` koşulsuz çalışır. Altı denemenin üçünde program takıldı, üçünde 10 üretilen elemandan yalnız 5'i tüketilip program bitti.

Hoca daha sonra kapasiteyi ve üretim sayısını değiştirerek birkaç deneme yaptı. Kapasite ile üretim sayısının birbirine karıştığını kendisi de söyledi ("Kafa karıştırdı burası"). Sayaç kapasiteye göre sınanınca, kapasite küçükken döngü erken bitiyordu ve üretim küçük sayılarda kalıyordu.

Bir öğrenci, dört çalıştırmanın ortak yönünü söyledi: Üretici bittikten sonra tüketici bir kere çalışıp duruyor. Sorun buradan sonra netleşmeye başladı.

1.6 Sonlandırma Sorunu: Dersteki Denemeler

Asıl sorun, tüketicinin döngüden ne zaman çıkacağıdır. Üretici son elemanı ekleyip sayacı 10 yapınca, tüketicinin `number < 10` koşulu yanlış olur. Listede eleman kalmış olsa bile tüketici döngüden çıkar. Çıktıda üretici 9'u üretmiş, tüketici geride kalmış görünür. Koşul `number <= 10` yapılınca da tüketici bitmedi ve main'e sıra gelmedi. Hoca bunu, `join` satırlarının altına eklediği "program bitti" yazısının ekrana gelmemesinden gördü.

Öneri: Bir öğrenci başka bir değişkene ihtiyaç olduğunu söyledi: `number` yalnız ne kadar üretildiğini tutuyor, ne kadar tüketildiğini de tutmak gerekir. Hoca bunu kabul etti, ama geçen yıl

bu problemi tek değişkenle çözdüğünü hatırladığını söyledi. Bu yüzden başka yollar denendi. Bir öğrenci de 10'dan başlayıp 0'a doğru sayan bir değişken önerdi.

Hoca ayrıca bu sayının sabit olmaması gerektiğine dikkat çekti: "Sen 10 tane üreteceksin, ben 10 tane tüketeceğim" diye bir şey yoktur. Üretim dinamik olabilir, tüketici de üretildiği kadar tüketmelidir. Mevzu aslında budur.

Liste boyuna dayalı denemeyi hoca şöyle değerlendirdi: `while (list.size() > 0)` yazılırsa tüketici üreticiden önce çalıştığında listeyi boş görür, döngüye hiç girmeden biter ve bir daha çağrılmaz. Hoca bunu görmek için döngü sonrasına bir mesaj ekledi. Şanslı durumda üretici önce çalışır, tüketici bir eleman alır, ama liste yine 0'a düşünce tüketici görevini bitirip çıkar. `do-while` ile çözülebileceği de söylendi, ama denenmedi.

Son olarak bir öğrenci, `while (list.size() == 0)` bekleme satırlarının `synchronized` bloğunun üstüne alınmasını önerdi. Bu denemede yine monitör hatası çıktı. Hoca sorunun bir yerdeki hatasından kaynaklanabileceğini, çözümü sonra göndereceğini söyleyerek GUI konusuna geçti.

> **Ders sonrası not:** Hoca bu hatada "çalışmamış bir thread üzerinde `wait` ile ilgili exception fırlatıyor" dedi; doğrusu, `wait()` kilidi tutmayan bir thread tarafından çağrılmıştır. Bekleme satırı `synchronized` bloğunun dışına alınınca bu durum oluşur. javac ile denendi: Liste boşken `t2 IllegalMonitorStateException: current thread is not owner` hatası verdi. Thread'in çalışmamış olmasıyla ilgisi yoktur.

Hoca thread konusunun çok daha detaylı olduğunu, işletim sistemleri dersinde görüleceğini söyledi.

2. Java Swing ile Pencere Oluşturma

2.1 GUI Kütüphaneleri ve Hiyerarşi

Hoca, iç sınıfları (inner class) daha önce anlattığı için tekrar etmedi. Java'da C#'tan farklı olarak birkaç GUI kütüphanesi vardır. İlk başta AWT vardı, sonra Swing çıktı. AWT çok eski bir sürümdür. Günümüzde JavaFX'in de farklı sürümleri vardır. Bunların hepsi olay güdümlü (event driven) bir yapıya dayanır.

Swing'in bir hiyerarşisi vardır. Etiket, buton ve tablo gibi bileşenler `JComponent`'ten gelir. Konteynerler (container) pencereyi mantıksal olarak bölmeye yarar. Çerçeveler (frame) ve diyaloglar da bu yapının üzerindedir. Hepsi sonuçta `Component` sınıfından kalıtım alır.

C#'ta form üzerinde çalışılıyordu. Swing'de bunun karşılığı `JFrame`'dir, buton `JButton`, metin kutusu `JTextField` olur. C#'ta `Form` sınıfından kalıtım alınmıştı. Burada da kalıtım alınabilir, ama hoca kalıtım almadan, doğrudan ilişkilendirme (association) ile yazdı: Sınıfın içinde bir `JFrame` nesnesi oluşturdu.

2.2 Pencerem Sınıfı

`Pencerem` adlı sınıfta `private JButton b` ve `private JTextField tf1` alanları tanımlandı. Sınıftan nesne üretildiğinde kurucu metot çağrılacağı için ilk değerler orada verildi. Kurucu metotta yapılan işler şunlardır:

1. `JFrame f = new JFrame();` ile pencere nesnesi oluşturulur.
2. `b = new JButton("Tıkla");` ile buton oluşturulur. Kurucuda isim verilebilir.
3. `b.setBounds(50, 100, 95, 30);` ile geometri ayarlanır. İlk iki sayı başlangıç noktasının x ve y koordinatıdır, sonraki ikisi genişlik ve yükseklikliktir. Yani buton (50, 100) noktasından başlar ve 95'e 30'luk bir alan kaplar.
4. `f.add(b);` ile buton pencereye eklenir. `add` bir `Component` alır. Buton bir `Component` olduğu için tür dönüşümü kendiliğinden olur.
5. `tf1 = new JTextField("");` ile metin kutusu boş olarak oluşturulur. Hoca C#'ta böyle bir kurucu olmadığını, bunun dilden dile değiştiğini, bileşenlerin kullanım kılavuzuna bakmak gerektiğini söyledi.
6. `tf1.setBounds(50, 50, 95, 30);` ile metin kutusu butonun üstüne yerleştirilir ve `f.add(tf1);` ile eklenir.

7. `f.setSize(300, 300);` ile pencere boyutu verilir, `f.setVisible(true);` ile pencere görünür yapılır.

`main` içinde `new Pencerem();` yazmak yeterlidir: Nesne üretilirken kurucu çağrılır ve pencere oluşur. İlk denemede `main` metodu `static` yazılmadığı için program çalışmadı, `public static void main` olarak düzeltildi.

2.3 setLayout(null)

Çalıştırınca pencere açıldı, ama buton verilen yerde görünmedi, yalnız bir metin kutusu görünüyordu. Hoca bir yerleşim (layout) parametresi olduğunu hatırladı ve `f.setLayout(null);` ekledi. Bundan sonra arayüz beklendiği gibi geldi.

> **Ek bilgi (derste anlatılmadı):** `JFrame`'in içerik bölmesi varsayılan olarak `BorderLayout` kullanır. Headless ortamda bir `JRootPane` üzerinden javac ile denendi: İçerik bölmesinin yerleşim nesnesi `BorderLayout`'tan geliyor. Aynı düzende 300'e 300'lük bir panele iki bileşen eklenince sonuncusu bütün alanı kapladı, öncekinin boyutu 0 kaldı ve `setBounds(50, 100, 95, 30)` değeri dikkate alınmadı. `setLayout(null)` kullanılınca bileşen (50, 100, 95, 30) değerlerinde kaldı.

2.4 Pencere Kapatma Davranışı

Hoca pencereyi çarpıya basarak kapattı, ama önceki çalıştırmaların da hâlâ açık olduğunu gördü: Pencere gidiyor, süreç (process) sonlanmıyordu. Süreçleri elle öldürdü. Java'da çarpı düğmesine görev atamak gerekir. Bunun için `setDefaultCloseOperation` vardır. Yardım penceresindeki seçenekleri okudu:

- `DO_NOTHING_ON_CLOSE` (0): Kapatmada bir şey yapma.
- `HIDE_ON_CLOSE` (1): Pencereyi gizle.
- `DISPOSE_ON_CLOSE` (2): Pencereyi dispose et, bellekle ilgili kısmı bitir.
- `EXIT_ON_CLOSE` (3): Çıkışta programı kapat.

Hoca önce parametreye 3 yazdı. Sabitin adını yazmayı deneyince bulunamadı. Bir öğrenci bunların `JFrame` içinde tanımlı olduğunu, `JFrame.EXIT_ON_CLOSE` yazılacağını söyledi. Hoca `System.out.println(JFrame.EXIT_ON_CLOSE)` ile ekrana 3 yazdırıp doğruladı. Artık pencere kapanınca süreç de kapanıyordu. Bu dört değer javac ile derlenen küçük bir programla da yazdırıldı: 0, 1, 2, 3.

Hoca bu pencereye daha fazla konteyner eklenebileceğini, görsel arayüzden fareyle sürükle bırak yapmanın da mümkün olduğunu söyledi.

> **Ders sonrası not:** Hoca bu sabitler için "makro" dedi; doğrusu, Java'da makro yoktur, bunlar `int` türünde sabit (`static final`) alanlardır. Okunabilirlik için 3 yerine `JFrame.EXIT_ON_CLOSE` yazmak daha doğrudur.

3. Butona Görev Atamak: İlk Olay Dinleyicisi

Hoca, C#'ta olayların kolay olduğunu, burada işin biraz daha zahmetli hale geldiğini söyledi. C# bu kodu kendisi oluşturuyordu, Java'da elle yazılır.

Buton için `addActionListener` metodu vardır. Buna `ActionListener` türünde bir nesne verilir. `ActionListener` Java AWT'den gelir. Bu nesne anonim sınıfla yazılabilir. Derleyici `actionPerformed` metodunun yazılmasını ister. Bir eylem (action) gerçekleştiğinde bu metot çalışır. Parametresi `ActionEvent` türündedir. C#'ta gördüğümüz `MouseEvent` gibidir, ama daha geniş bir olaydır.

Yapılacak iş bu metodun içine yazılabilir. Ama hoca, her şey yukarıya yazılırsa oranın çok karıştığını söyleyerek `private void button_click(ActionEvent e)` adında ayrı bir metot yazdı ve `actionPerformed` içinden onu çağırdı.

Metin kutusuna yazmak için `tf1.setText(...)` kullanılır. C#'taki gibi doğrudan `Text` özelliği yoktur. Sonra ikinci bir metin kutusu eklendi: `tf2`, içeriği "Merhaba". Konum hesabı derste şöyle yapıldı:

- **Verilen:** `tf1` x = 50'den başlıyor, genişliği 95.
- **İşlem:** $50 + 95 = 145$. `tf1`'in sağ kenarı 145'tir.
- **Sonuç:** Araya biraz boşluk girsin diye `tf2` x = 150, y = 50 noktasından başlatıldı. Kutunun sağ kenarı $150 + 95 = 245$ olur, 300 genişliğindeki pencereye sığar (ara adım eklendi). Python ile hesaplandı.

Hoca önce `tf2` için kopyaladığı kodu çalıştırdı, ikinci kutu görünmedi. `f.add` satırında hâlâ `tf1` yazdığını görüp `tf2` yaptı.

Tıklama sayısını da yazdırmak için bir sayaç gerekiyordu. Hoca `button_click` içine `static int value = 1;` yazdı ve "illegal start of expression" hatası aldı. Önce nedenini anlayamadı, sonra değişkeni sınıf alanına taşıdı ve 0 ile başlattı. Sonuçta `tf2`'deki metin alınıp `tf1`'e yazdırıldı: İlk tıklamada "Merhaba, butona 1 kez tıklandı", ikincisinde "Merhaba, butona 2 kez tıklandı".

> **Ders sonrası not:** Hoca hatanın nedenini derste bulamadı; doğrusu, Java'da metot içinde `static` yerel değişken tanımlanamaz. javac ile denendi, aynı "illegal start of expression" hatası alındı. Sayaç sınıf alanı olmalıdır.

Derste yazılan kodun toparlanmış hali aşağıdadır. Kod javac ile derlendi. Ekranda pencere açılmadı; butonun davranışı headless ortamda, `JFrame` kullanılmadan `doClick()` ile denendi ve yukarıdaki iki metin elde edildi.

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JTextField;

public class Pencerem {
    private JButton b;
    private JTextField tf1, tf2;
    private static int value = 0;

    public Pencerem() {
        JFrame f = new JFrame();

        b = new JButton("Tıkla");
        b.setBounds(50, 100, 95, 30);
        f.add(b);

        b.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                button_click(e);
            }
        });

        tf1 = new JTextField("");
        tf1.setBounds(50, 50, 95, 30);
        f.add(tf1);

        tf2 = new JTextField("Merhaba");
        tf2.setBounds(150, 50, 95, 30);
        f.add(tf2);

        f.setSize(300, 300);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }

    private void button_click(ActionEvent e) {
```

```
        value++;
        tf1.setText(tf2.getText() + ", butona " + value + " kez tıklandı");
    }

    public static void main(String[] args) {
        new Pencerem();
    }
}
```

> **Ek bilgi (derste anlatılmadı):** Swing bileşenleri büyük olasılıkla thread güvenli değildir. Bu yüzden pencereyi oluşturan kod büyük olasılıkla olay thread'inde (Event Dispatch Thread) çalıştırılmalıdır: `SwingUtilities.invokeLater(() -> new Pencerem());`. Derste `main` içinde doğrudan `new Pencerem();` yazıldı.

4. Aradan Sonra: Üretici-Tüketici Koduna Dönüş

4.1 Kapasite ve max Parametresi

Aradan sonra hoca, kodda ilk bakışta bir sıkıntı görünmediğini söyledi ve kapasiteyi 5 yaptı. Listenin boyu 5'ten fazla olmayacaktır. Üretilecek eleman sayısı ise 10'dur. Hoca bunu özellikle böyle seçti: 10 tane üretilecek, ama tamponun kapasitesi 5 olduğu için arada tüketim olacak, üretim öyle devam edecektir.

Üretim sayısının ayarlanabilir bir parametre olmasını da istedi. Bunun için sınıfa bir `int max` alanı ve `this.max = max` yazan bir kurucu ekledi. Koddaki 10 yerine `max` kullanıldı, test sınıfında nesne `new PC(10)` ile üretildi.

4.2 Yeni Denemeler

Çalıştırınca üretici 9'u üretti, tüketici 8'de kaldı. Hoca "yani bir geride kalıyor" dedi. Bir sonraki çalıştırmada ise program sorunsuz çalıştı, 9 da tüketildi. Bir öğrenci, programın şanslı olunduğunda çalıştığını söyledi: Üretici 8'i ürettikten sonra tüketici hemen 8'i tüketirse sorun çıkmıyor.

Öneri: Bir öğrenci sorunu şöyle açıkladı: Üretici son elemanı üretince sayaç `max`'a eşit olur. Tüketici ondan sonra dönerse `number < max` yanlış olduğu için döngüye girmez. Koşul `<=` yapılırsa çözülür diye tahmin etti. Hoca denedi. Bu kez tüketici son elemanı aldı, ama döngüye yine girip beklemeye geçti ve program bitmedi.

Hoca `<=` ile birlikte `list.size() > 0` koşulunu düşündü. Sonra bir öğrenci, `wait` satırındaki koşula "üretim sayısı `max`'a eşit değilse" eklenmesini önerdi: `while (list.size() == 0 && number != max)`. Bu denemede de son elemanlar tüketilmedi. Yukarıdaki `number < max` koşulu tüketici tarafında kalmıştı. Bir öğrenci bunu söyleyince koşul `<=` yapıldı. Bu kez `removeFirst` için "böyle bir eleman bulunamadı" hatası çıktı: Tüketici boş listeden bir kez fazla almaya çalışıyordu.

> **Hocanın vurgusu:** Thread işleri algoritmik düşüncenin de ötesinde biraz daha karışıktır. Üretici-tüketici meşhur ve kafa yoran bir problemdir. Hoca, doğru kodu paylaşacağını söyledi ve "biraz da siz uğraşın" dedi.

Dersteki denemeler aynı koşullarla javac ile derlenip çalıştırıldı. Kapasite 5, `max` 10 alındı. Denemeleri kısaltmak için bekleme 500 ms yerine 20 ms yapıldı, 3 saniyede bitmeyen çalıştırma "takıldı" sayıldı.

Tüketici kodu	Derste görülen	javac ile deneme sonucu
`while (number < max)` ve `while (size == 0) wait();`	Tüketici geride kaldı, bazen sorunsuz bitti	20 denemenin 9'unda 9 eleman tüketildi, 1 eleman kaldı; 11'inde 10 eleman tüketildi
`while (number <= max)` ve `while (size == 0) wait();`	Tüketici bitmedi, main'e sıra gelmedi	9 denemenin hepsinde takıldı: Üretici bitti, tüketici `wait()` içinde kaldı, onu uyandıracak kimse yok

Tüketici kodu	Derste görülen	javac ile deneme sonucu
`while (size > 0)`	Tüketici erken çıktı	9 denemenin hepsinde takıldı: Tüketici 0 ya da 1 eleman alıp çıktı, üretici dolu listede bekledi
`number` tüketicide de azaltılıyor	Hoca “hiç bitmez” dedi	9 denemenin hepsinde program bitmedi
Bekleme satırı `synchronized` dışında	Monitör hatası	`IllegalMonitorStateException`
`while (number < max)` ve `while (size == 0 && number != max) wait();`	Son elemanlar tüketilmedi	20 denemenin 8’inde 9 eleman tüketildi, 12’sinde 10 eleman tüketildi
`while (number <= max)` ve `while (size == 0 && number != max) wait();`	“Eleman bulunamadı” hatası	6 denemenin hepsinde `NoSuchElementException`

Sonuçların bir kısmının çalıştırmadan çalıştırmaya değişmesi derste görülenle aynıdır: Sonuç, thread’lerin hangi sırayla çalıştığına bağlıdır.

4.3 Çalışan Sürüm

> **Ders sonrası not:** Derste kod tam olarak düzeltilemedi. Aşağıdaki birinci sürüm, derste bir öğrencinin önerdiği gibi tüketilen eleman için ayrı bir sayaç kullanır. Bu sürüm dersin kaydında yazılmadı, kısa nottaki çözümden kurulmuştur. Üretici ve tüketici bitip bitmediğine `synchronized` içinde bakar.

```
import java.util.LinkedList;

public class PC {
    private final LinkedList<Integer> list = new LinkedList<>();
    private final int capacity = 5;
    private final int max;
    private int uretimSayisi = 0;
    private int tuketimSayisi = 0;

    public PC(int max) {
        this.max = max;
    }

    public void produce() throws InterruptedException {
        int value = 0;
        while (true) {
            synchronized (this) {
                if (uretimSayisi == max) {
                    return;
                }
                while (list.size() == capacity) {
                    wait();
                }
                System.out.println("Üretici üretti: " + value);
                list.add(value++);
                uretimSayisi++;
                notify();
            }
            Thread.sleep(500);
        }
    }

    public void consume() throws InterruptedException {
        while (true) {
```

```
            synchronized (this) {
                if (tuketimSayisi == max) {
                    return;
                }
                while (list.size() == 0) {
                    wait();
                }
                int val = list.removeFirst();
                tuketimSayisi++;
                System.out.println("Tüketici tüketti: " + val);
                notify();
            }
            Thread.sleep(500);
        }
    }

    public static void main(String[] args) throws InterruptedException {
        PC pc = new PC(10);
        Thread t1 = new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    pc.produce();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });
        Thread t2 = new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    pc.consume();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Program bitti");
    }
}
```

Bu kod javac ile derlenip çalıştırıldı: 10 eleman üretildi, 10'u da 0'dan 9'a sırayla tüketildi, en sonda "Program bitti" yazdı. Ayrıca bekleme süreleri 1 ms ile 7 ms arasında değiştirilerek (üretici hızlı, tüketici hızlı, ikisi de hızlı) 50 elemanla 45 kez çalıştırıldı. Hepsinde 50 üretim ve 50 tüketim oldu, tüketim sırası korundu, liste kapasiteyi aşmadı.

Hoca geçen yıl bu problemi fazladan değişken olmadan çözdüğünü, ayrıca tüketicinin kaç eleman üretileceğine sabit bir sayıyla bağlı olmaması gerektiğini söylemişti. Bu iki istek de karşılanabilir. Tüketicinin döngüden çıkma kararı, listenin boş olduğu anda verilir: Liste boşsa ve üretim bittiyse çık, üretim bitmediyse bekle. Yalnız `consume` metodu ve alanlar değişir:

```
import java.util.LinkedList;

public class PC2 {
    private final LinkedList<Integer> list = new LinkedList<>();
    private final int capacity = 5;
```

```
private final int max;
private int number = 0;

public PC2(int max) {
    this.max = max;
}

public void produce() throws InterruptedException {
    int value = 0;
    while (true) {
        synchronized (this) {
            if (number == max) {
                return;
            }
            while (list.size() == capacity) {
                wait();
            }
            System.out.println("Üretici üretti: " + value);
            list.add(value++);
            number++;
            notify();
        }
        Thread.sleep(500);
    }
}

public void consume() throws InterruptedException {
    while (true) {
        synchronized (this) {
            while (list.size() == 0) {
                if (number == max) {
                    return;
                }
                wait();
            }
            int val = list.removeFirst();
            System.out.println("Tüketici tüketti: " + val);
            notify();
        }
        Thread.sleep(500);
    }
}

public static void main(String[] args) throws InterruptedException {
    PC2 pc = new PC2(10);
    Thread t1 = new Thread(() -> {
        try {
            pc.produce();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
    Thread t2 = new Thread(() -> {
        try {
            pc.consume();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
    t1.start();
```

```
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Program bitti");
    }
}
```

Bu sürüm de aynı denemelerle javac ile sınandı, 45 çalıştırmanın hepsi tamamlandı. Takılma olmaz, çünkü tüketici `wait()`'e yalnız üretim bitmemişken girer ve üretici her üretimden sonra `notify()` çağırır. Burada thread'ler anonim sınıf yerine lambda ifadesiyle yazıldı. Bu gösterim derste kullanılmadı, ikinci sürümde yer kazanmak içindir.

5. Olay Yönetiminin Ayrıntıları

5.1 GUI Kodu Neden Karışıyor?

Hoca, GUI'de şimdiye kadar yeni bir nesneye yönelik kavram görmediğimizi söyledi. Yapılan işler dersin başında görülen kalıtım, ilişkilendirme, referans ve ilklendirmedir. Ama işe yazılım mühendisliği açısından da bakmak gerekir. Çünkü gerçek bir pencerede veri tabanından veri getirilip geri kaydedilecek, belki bir algoritma çalışacaktır (bir matris çarpımı ya da veri madenciliği algoritması gibi). Bir de grafik arayüz kodu vardır.

> **Hocanın vurgusu:** Veri tabanı kodu, algoritma ve grafik arayüz kodu hepsi bir arada olunca çok kötü görünür. Bu konuya katmanlı mimaride dönülecektir.

5.2 Olay ve Dinleyici İlişkisi

Bir eylem gerçekleştiğinde çalışacak metot, dinleyici üzerinden belirtilir. `ActionEvent`, hangi nesnede olay gerçekleştiyse onun bilgilerini getirir. C#'ta bu iş üst sınıfta doğrudan halledilir. Java'da daha ayrıntılıdır, çünkü her olay türünün kendi dinleyicisi vardır:

- Fare olayları: `MouseListener` ve `MouseMotionListener` ayrı ayrı çalışır.
- Fare tekerleği olayları: `MouseWheelListener`.
- Klavye olayları: `KeyListener`.

Olay yönetiminde (event handling) önce bileşen ilgili dinleyiciyle kaydedilir:

- Buton ve menü öğesi: `addActionListener`. Eylem dinleyicisi çoğunlukla tıklamaya benzer bir olay içindir.
- Metin kutusu (`JTextField`): Yine `addActionListener`. Kutunun içinde klavyeden bir tuşa basılınca oluşan eylemdir.
- Onay kutusu (checkbox) ve seçim kutusu (choice): `addItemListener`. Olay türü `ItemEvent`'tir.

Hoca aynı şeyin metin alanında (text area) da olabileceğini söyledi.

> **Ders sonrası not:** Hoca metin alanı için de eylem dinleyicisi olabileceğini söyledi; doğrusu, Swing'deki `JTextArea`'da `addActionListener` yoktur. javac ile denendi, "cannot find symbol" hatası alındı. `JTextField`, `JButton`, `JMenuItem`, `JCheckBox` ve `JComboBox`'ta ise bu metot vardır. Hocanın metin kutusu için söylediği eylem de büyük olasılıkla her tuşta değil, yalnız Enter tuşunda oluşur.

Ardından olayı işleyen kod yazılır. Hoca bunun üç yeri olduğunu anlattı: sınıfın kendisi, dış bir sınıf ve anonim sınıf.

5.3 Birinci Yol: Sınıfın Kendisi (this)

Sınıf `ActionListener` arayüzünü uygular, `actionPerformed` metodunu yazar ve butona `b.addActionListener(this)` der. Hoca bunun en kolay yollardan biri olduğunu söyledi.

> **Ders sonrası not:** Hoca "`this` diyerek bu sınıfa ait bütün nesnelerin özellikleri gönderiliyor" dedi; doğrusu, `this` bu nesnenin dinleyici olarak kaydedilmesidir. Olay oluştuğunda bu nesnenin `actionPerformed` metodu çağrılır, olayın bilgisi `ActionEvent` parametresiyle gelir.

İki buton olduğunda ve ikisine de `this` verildiğinde iki butonun olayları da aynı metoda gelir. O zaman önce hangi butona basıldığı bulunmalıdır. Hoca `ActionEvent` içinde `getClass`, `getID`, `getSource` gibi metotlar olduğunu gösterdi: `e.getSource() == b1` ise birinci butona, `e.getSource() == b2` ise ikinci butona basılmıştır. Çalıştırınca "birinci butona basıldı", "ikinci butona basıldı" mesajları geldi.

Derste ikinci buton eklendiğinde önce tepki vermedi, çünkü dinleyici yalnız `b` için yazılmıştı. Hoca: "`b2` için dinleyiciyi yazmayınca bu hiçbir şeye yaramaz." Dinleyici eklenince çalıştı.

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;

public class IkiButon implements ActionListener {
    private JButton b1, b2;

    public IkiButon() {
        JFrame f = new JFrame();
        b1 = new JButton("Birinci");
        b1.setBounds(50, 100, 95, 30);
        b1.addActionListener(this);
        f.add(b1);

        b2 = new JButton("İkinci");
        b2.setBounds(150, 100, 95, 30);
        b2.addActionListener(this);
        f.add(b2);

        f.setSize(300, 300);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == b1) {
            System.out.println("Birinci butona basıldı");
        } else if (e.getSource() == b2) {
            System.out.println("İkinci butona basıldı");
        }
    }

    public static void main(String[] args) {
        new IkiButon();
    }
}
```

Hoca, bu kolay görünen yolun arkasından başka işler çıktığını, ama bazı durumlarda ortak (global) bir metodun işe yaradığını anlattı.

> **Hocanın vurgusu:** Bir stadyumda 40.000 koltuk ve her koltuk için bir buton olduğunu düşünün. 40.000 metot yazacak değiliz. Mecburen ortak bir metot yazacağız ve hangi butona basıldığını onun içinde bulacağız.

Soru: Butonlara ayrı ayrı değil de forma tek bir dinleyici eklesek olur mu? Formda yapılan bütün işlemleri yakalasa? **Cevap:** Form başlı başına bir nesnedir. Hoca `f.addActionListener` yazmayı denedi, böyle bir metot çıkmadı. Listede `ComponentListener` ve `ContainerListener` olduğunu gördü, bunlardan biri olabileceğini, ama ayrıntısını bilmediğini söyledi. Bu, buton dinleyicisinden tamamen farklıdır.

> **Ders sonrası not:** Hoca "muhtemelen component ya da container listener" dedi.

javac ile denendi: `JFrame`'de `addActionListener` yoktur, `addWindowListener`, `addComponentListener` ve `addContainerListener` vardır. Pencere olayları (kapatma gibi) büyük olasılıkla `WindowListener` ile, boyut ve konum değişimi `ComponentListener` ile, konteynere bileşen eklenip çıkarılması `ContainerListener` ile dinlenir. Bunların hiçbiri büyük olasılıkla butona basılma olayını forma getirmez.

Soru: Her buton için ayrı ayrı bunları oluşturmadan çözmenin yolu yok mu? **Cevap:** Butonlar dizi olarak tanımlanır. Hoca iki elemanlı bir diziyle gösterdi ve birden çok buton varsa `for` döngüsüyle tek seferde yapılabileceğini söyledi.

Dizi tanımında hoca önce yanlış bir yazım denedi, C# ile karıştırdığını söyledi. Doğrusu önce referans dizisini tanımlamak, sonra `new JButton[100]` ile oluşturmaktır. Sonra `B[i]` ile `i`. elemana döngüyle erişilir. Derste butonlardan biri küçük `b` olarak kalınca "array required" hatası alındı, dizi adı büyük `B` yapılarak düzeltildi. Aynı hata javac ile denendi: `array required, but JButton found`.

> **Ders sonrası not:** Hoca `new JButton[100]` için "100 elemanlık bir JButton referansı oluşuyor" dedi. Bu, 100 tane boş (`null`) referans demektir, buton nesnesi oluşmaz. javac ile denendi: Oluşturulmamış bir eleman `null` çıktı ve metot çağrısı `NullPointerException` verdi. Her eleman ayrıca `new JButton(...)` ile oluşturulmalıdır.

> **Ders sonrası not:** Hoca, `getSource`'tan "buton 1" gibi bir şey geleceğini ve bunun string işlemleriyle ayrıştırılacağını söyledi; doğrusu, `getSource()` olayı oluşturan nesnenin kendisini döndürür. javac ile denendi: Dönen nesnenin türü `javax.swing.JButton`'dır. Hangi buton olduğu, aşağıdaki gibi referansları döngüyle karşılaştırarak bulunur. Hocanın bahsettiği ID için de `setActionCommand` ile verilen metin `getActionCommand` ile okunabilir; bu da javac ile denendi.

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;

public class ButonDizisi implements ActionListener {
    private JButton[] B;

    public ButonDizisi() {
        JFrame f = new JFrame();
        B = new JButton[2];
        for (int i = 0; i < B.length; i++) {
            B[i] = new JButton("Buton " + i);
            B[i].setBounds(50 + i * 100, 100, 95, 30);
            B[i].addActionListener(this);
            f.add(B[i]);
        }
        f.setSize(300, 300);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        for (int i = 0; i < B.length; i++) {
            if (e.getSource() == B[i]) {
                System.out.println(i + ". butona basıldı");
            }
        }
    }

    public static void main(String[] args) {
        new ButonDizisi();
```

```
    }
}
```

Kod javac ile derlendi. Ekranda pencere açılmadı. Döngüdeki karşılaştırma headless ortamda `doClick()` ile denendi ve doğru sıra numarası yazıldı.

Hoca web tarafında bu işin daha kolay olduğunu da ekledi: HTML etiketlerinin `name`, `value`, `class`, `id` gibi nitelikleri vardır, olaylar bunlarla daha hızlı çözülür. Burada ise dizi mantığıyla daha statik bir çözüm kurulur.

5.4 İkinci Yol: Dış Sınıf

`ActionListener`'ı uygulayan ayrı bir sınıf yazılır, bu sınıfın nesnesi `addActionListener`'a verilir. Geri kalan her şey aynıdır. Hoca bunun çok fazla sınıf ortaya çıkardığını, bu yüzden pek tercih edilmediğini söyledi.

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;

class Tiklama implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Butona basıldı");
    }
}

public class DisSinif {
    public DisSinif() {
        JFrame f = new JFrame();
        JButton b = new JButton("Tıkla");
        b.setBounds(50, 100, 95, 30);
        b.addActionListener(new Tiklama());
        f.add(b);
        f.setSize(300, 300);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }

    public static void main(String[] args) {
        new DisSinif();
    }
}
```

Bu kod derste yazılmadı, hocanın slaytta anlattığı yolun Bölüm 3'teki butona uygulanmış halidir. javac ile derlendi, ekranda pencere açılmadı.

5.5 Üçüncü Yol: Anonim Sınıf

Bu, Bölüm 3'te yazılan yoldur: `addActionListener` içinde `new ActionListener() { ... }` yazılır. Hoca görüntüsünü "çirkin" buldu, ama işi gördüğünü söyledi. İkisi birleştirilebilir: Anonim sınıf yalnız `button_click` metodunu çağırır, asıl kod o metotta durur. Birden çok buton varsa aynı kodu iki kez yazmak gerekmez, hangi butondan geldiğine metodun içinde bakılır. Genelde 100-200 buton olduğunda döngüyle kontrol etmek daha mantıklıdır.

6. Temel Bileşenler ve Python'daki Karşılığı

`JButton` kurucusu boş bırakılırsa buton üzerinde bir şey olmaz, `String` verilirse yazı konur, ikon verilirse ikon konur. Çok kullanılan metotlar `set` ve `get` metotlarıdır: Üzerindeki yazı ayarlanır,

buton etkin ya da pasif (enable, disable) yapılır, ikon verilir. `addActionListener` butonu eyleme bağlar.

Etiketler (label) birçok yerde karşımıza çıkar. Hoca Zoom'daki katılımcı penceresindeki yazıları örnek gösterdi. Etiket bir anlamda `System.out.println`'in yaptığı işi yapar, yani bir şey gösterir. `getText` ve `setText` ile okunur ve yazılır, yatay ve dikey hizalaması ayarlanır, ikonlu da olabilir. Genelde kullanıcı etiketin üzerine yazmaz, etiket yalnız gösterir. Metin kutusu (text field) ise içine yazı yazılan bileşendir.

Python'da aynı iş PyQt kütüphanesinde `connect` ile yapılır. Hoca iki yol gösterdi. Birincisinde nesne üzerinden yazılır: `Button1.clicked` olayı `connect` ile bir tıklama metoduna bağlanır. İkincisinde daha genel bir yolla `QObject` üzerinden, `b2` butonuna tıklama sinyali geldiğinde `b2` tıklama metodu çağrılır. İkisi de aynı işi yapar.

Hoca, her dil için çok sayıda GUI kütüphanesi olduğunu, Wikipedia'da "GUI libraries" diye aratılabileceğini, hangisi çok kullanılıyorsa onunla devam edilebileceğini söyledi.

7. Katmanlı Mimari

Gerçek uygulamalarda üç tür iş vardır: veri tabanı işlemleri, kullanıcı arayüzü işlemleri ve iş mantığı (business logic), yani algoritmik işlemler. Araya bağlantılar ve kontroller de girer. Hepsini tek bir sınıfta ya da karışık sınıflarda yazmak makul değildir. Bunun yerine katmanlı mimari (layered architecture, n-tier architecture) kullanılır. Hocaya göre bu bir anlamda bir metodolojidir.

Katmanlar şöyle ayrılır:

- **Veri tabanı katmanı:** Veri tabanıyla ilgili işlemler bir sınıf grubunda yazılır: bağlantı kurma, kayıt, veri getirme ve götürme, yani ekleme, silme, güncelleme gibi CRUD işlemleri. Bu katman veri tabanıyla doğrudan iletişim halindedir. Örneğin bir öğrenci sınıfında ID ve isim tanımlanır.
- **Kullanıcı arayüzü:** Butonların ve metin kutularının eklendiği yerdir. Arayüze veri gelir ve gider, ama arayüz bu verinin veri tabanından gelip gelmediğini bilmez. O yalnız bir arayüzdür.
- **İş mantığı katmanı (business logic layer):** Gelen ve giden verileri kontrol etmek için eklenen ara katmandır.
- **Façade katmanı:** Veri tabanı işlemleriyle arayüz arasındaki bağlantıyı kurar. Hoca kelimenin Fransızcadan geldiğini ve "vitrin" olduğunu söyledi. Arayüzden gelen ve iş mantığında kontrol edilen veriyi veri tabanına aktaran komutlar burada bulunur. Ters yönde de veri tabanından gelen bir liste bu katman üzerinden grafik arayüze gönderilir.

Bunların hepsi sınıftır. Her birini bir Java dosyası gibi düşünebiliriz. Ayrı bir kütüphane olarak yazmak da mümkündür.

Hoca bu yapıyı karpuz taşımaya benzetti: 100 tane karpuz taşınacak. İnsanlar karpuzları birbirine atar. Atmazlarsa herkesin karpuzu alıp yürümesi gerekir. Yürümenin maliyetine gerek yoktur, kişi durduğu yerde çok sayıda karpuzu iletebilir. Hem yürüme maliyetinden kurtulmak hem de herkesin kendi işini daha modüler yapması için uygulama katmanlara bölünür.

> **Ek bilgi (derste anlatılmadı):** Façade, büyük olasılıkla tasarım kalıpları arasında da aynı adla geçer. Bu kalıpta karmaşık bir alt sistem, tek ve basit bir arayüzün arkasına saklanır. Derste anlatılan façade katmanı da arayüzün veri tabanı ayrıntılarını bilmemesini sağlar.

8. MVC (Model-View-Controller)

8.1 Katmanlarla Karşılığı

Hoca, katmanlı yapının tasarım kalıplarında daha güncel kullanılan halinin MVC olduğunu söyledi. Karşılıkları şöyledir:

- **Model:** Veri tabanı tarafına karşılık gelir.
- **View:** Kullanıcı arayüzüne karşılık gelir.

- **Controller:** Façade ve iş mantığı katmanının yaptığı işi yapar. Veri tabanına gelen giden verilerin arayüzde görünmesi için kontrolle işlem yapılır.

Herkesin aklına MVC deyince web gelse de bu yapı masaüstü uygulamalarında da vardır, çünkü bir tasarım kalıbıdır.

> **Hocanın vurgusu:** Bu dersten sonra tasarım kalıplarını (design patterns) almanız ya da öğrenmeniz iyi olur. Piyasa ve yazılım mühendisliği bu yönde ilerliyor. Model-View-Controller onlardan bir örnektir.

8.2 Model

Hoca C# ile yazılmış bir ASP.NET MVC örneği gösterdi. Model, veri tabanındaki sütunların birebir kopyasıdır. Veri tabanında bir öğrenci tablosu varsa birebir onun sınıfı yazılır: öğrencinin ID'si, ismi, yaşı.

Model şu yüzden kullanılır: Veri tabanında çok sayıda tablo, tabloların altında çok sayıda sütun olabilir. Bunların hepsini tek tek `int` ve `string` gibi temel veri tipleriyle işlemek çok uzun sürer. Onun yerine nesnelerle veri transferi yapılır. Böylece ilkel veri takıntısından (primitive obsession) kurtulunur. Veri tabanında 100 tablo varsa her biri için bir sınıf olur, tablonun her sütunu da o sınıfın bir üyesidir.

8.3 View

View, web sayfasında ya da grafik uygulamada gördüğümüz şeydir. Arkasında HTML kodları vardır, ama bu kodlar çalışılan programlama diliyle programlanır. Java'da Spring Framework ile Java, burada C# kullanılır. Visual Basic ile yazılacaksa VBHTML, C# ile yazılacaksa CSHTML olur, eskiden ASPX vardı.

View ile Controller birbirine bağlıdır. Hoca, üniversitenin web sitesine girince gelen ana sayfayı örnek verdi: Her sayfanın kendine ait bir denetleyicisi olur, ana sayfanın (homepage) da home controller'ı vardır. Öğrenci eklemek ya da ders eklemek istediğinizde yeni bir sayfa açılır, her biri için ayrı bir web sayfası yazılır.

Sayfanın kodunda HTML etiketleri (paragraflar, tablolar, `td`) ile `@` ile başlayan C# kodları bir aradadır. Bu gösterime Razor sözdizimi (Razor syntax) denir. Sayfaya model olarak bir öğrenci dizisi gelir, her öğrencinin adı, ID'si ve soyadı ekrana yazdırılır. HTML helper'lar bu işi kolaylaştırır.

Hoca ekrandaki tablo için sordu: Bu veriler nereden geldi? Esasen veri tabanından geldi. Ama View, verinin veri tabanından geldiğini bilmez. Ona yalnız modelden, controller aracılığıyla geldiğini bilir.

8.4 Controller ve Üçünün Birleşmesi

Kullanıcı öğrenci listesini istediğinde akış şöyledir:

1. Kullanıcı öğrenciler sayfasına girer. Bu sayfanın ilk giriş sayfası `Index`'tir.
2. İstek `StudentController` sınıfına gelir. Sınıfın adı `Student` kısmı ve sonuna eklenen `Controller` ile oluşur, sınıf `Controller` sınıfından kalıtım alır.
3. Controller veri tabanından modele veri çeker. Modeli bir nesne gibi düşünürsek 10 ya da 100 öğrenci bir `ArrayList` ya da `LinkedList` içinde tutulabilir.
4. Controller `View` çağrısıyla listeyi `Index` sayfasına gönderir. `View` çağrısı adın başına `Student` ekleyerek `Student` altındaki `Index` sayfasını açar.
5. Sayfa, gelen modeldeki kayıt sayısı kadar satırı ekrana yazar.

Derste gösterilen örnekte liste elle yazılmış statikti, veri tabanından gelmiş gibi duruyordu. Hoca buna repository dedi. Gerçek uygulamada bu satır şuna benzer: `db` üzerinden ID'si 4 olan öğrenciyi getir, ardından `return View(...)` ile sayfaya gönder. Ya da bir ders ID'si verilir ve dersi seçen öğrencilerin listesi gelir.

Böylece proje üçe bölünmüş olur, herkesin yeri ayrıdır: View'ın içine HTML ile ilgili şeyler, Controller'ın içine başka şeyler yazılır.

Hoca sonra ileri konuları saydı. Yönlendirme (routing) mekanizmaları vardır. `ViewBag`, `ViewData` ve `TempData` parametre aktarımı içindir. Çünkü Controller'dan View'a tek bir nesne gider, araya virgül koyup iki nesne gönderilemez. Birden çok sorgu sonucu gerekiyorsa ya nesneler birleştirilip bir üst sınıftan yeni bir sarmalayıcı (wrapper) oluşturulur ya da `ViewBag` gibi mekanizmalar kullanılır. Bunun için biraz HTML bilgisi de gerekir. Hoca bu konuları başka bir dersinde uzun uzun anlattığını söyledi.

9. MVP (Model-View-Presenter)

Dersin bu kısmını bir öğrenci sunum olarak anlattı, hoca araya girerek ekleme yaptı.

9.1 Neden Ortaya Çıktı?

Sunuma göre MVC 80'lerin sonunda ortaya çıkmış bir mimari desendir. Büyük bir sorunu çözmüştür, ama kendisi de bazı sorunlar çıkarır: Görüntü ile model arasındaki bütün işlemleri Controller yaptığı için Controller'ın görevleri ve satır sayısı çok büyüyebilir. Piyasada bunun için "massive view controller" deyimi kullanılır.

Bunu çözmek için MVP ortaya çıkmıştır. Sunumda bunun Apple ve IBM'in içinde bulunduğu ortak bir şirkette doğduğu söylendi. MVC'nin üzerine inşa edilir, Controller'ı Presenter ile değiştirir. Presenter'ın görevleri Controller'a benzer, ama daraltılmıştır.

> **Ders sonrası not:** Sunumda MVC için "80'lerin sonu" dendi; doğrusu büyük olasılıkla 1970'lerin sonudur (Trygve Reenskaug, Smalltalk). Sunumda şirket adı altyazıda "Intelligent Inc." diye geçti; büyük olasılıkla Taligent kastedilmiştir ve MVP büyük olasılıkla 1990'ların başındandır.

9.2 Parçalar

MVC'deki ikiye ayırma (veri ve görüntü) burada da vardır. İlk tanımında desen altı parçaya bölünür. Veri tarafında üç parça vardır:

- **Model:** Veri tabanındaki bilginin yapısını oluşturur. Veriyi alıp getiren bazı manager sınıfları da olabilir. Örneğin bir API'den JSON verisi çekip modele çeviren sınıflar genelde model tarafına alınır. Bazıları bunları Presenter tarafına koyar, burası tam belirlenmiş değildir.
- **Selection:** Gelen veriden hangilerinin alınacağını belirler. Presenter'dan alınmış küçük bir görevdir.
- **Command:** Kaydetme, silme gibi işlemleri yapan ayrı sınıftır. Presenter bu sınıfın metotlarını çağırır. Böylece Presenter'ın satır sayısı küçülür.

Görüntü tarafında üç parça daha vardır:

- **View:** Butonlar ve diğer bileşenler.
- **Interactor:** Kullanıcı girdisini alır ve Presenter'a iletir.
- **Presenter:** İki taraf arasındaki köprüdür.

Interactor, Selection ve Command arada kalmış parçalardır. İlk tanımda vardırlar, ama günlük kullanımda çoğunlukla yoktur.

Presenter'ın görevi veriyi görüntüye uygun hale getirmektir. Sunumdaki örnek: Veri tabanından doğum tarihi gelir, ama ekranda yaş gösterilecektir. Bu çevirme Presenter'da yapılır.

Soru: (Hoca sordu) Bu, bahsedilen iş mantığı aslında. Gelen verinin amaca uygun işlenmesi ya da kaydedilecek verinin uygunluğunun kontrolü. Örneğin kitap adı girerken alanı boş bırakırsanız ya da boşluk girerseniz bu kayıt veri tabanına eklenmemelidir. Bu kontrol MVP'de burada mı yapılıyor? **Cevap:** Evet. Kaydedilecek ya da gösterilecek veri üzerindeki bütün değişim işlemleri Presenter'da yapılır.

Hoca bunun iş mantığı adıyla önceden de bilindiğini, bu teknolojilerin geçmişten geleceğe sürekli geliştiğini söyledi. Bir matris çekecekseniz matrise uygun verilerin gelip gitmesi beklenir. Öğrenci de Presenter'ın iş mantığı ile görüntü mantığının tam arasında oturduğunu, asıl görevinin aradaki geçişi sağlamak olduğunu ekledi.

9.3 Veri Akışı ve Bağımlılık Döngüsü

1. Kullanıcı bir butona basar. Bu bilgi (varsa Interactor aracılığıyla) Presenter'a gider.
2. Presenter girdiye göre yapılacak işi çağırır: Veri tabanına kaydetme ya da veri çekme.
3. İş bitince Presenter sonucu görüntüye uyarlar.
4. Görüntü, Presenter'dan haberi alır, veriyi alır ve kendini gösterir.

Burada bir döngü oluşmamalıdır. Presenter, veri tabanıyla konuşan bir manager sınıfına veri gönderir. Manager da Presenter'a referans tutarsa bir bağımlılık döngüsü (dependency cycle) oluşur. Bunu kırmak için bildirimler (notification), Swift'te closure yapıları ya da delegation kullanılır. Örneğin manager veriyi kaydettikten sonra "kaydettim" bilgisini güçlü bir referansla değil, zayıf bir bağ ile geri gönderir.

9.4 Avantaj ve Dezavantajlar

- **Test edilebilirlik:** MVC'de bütün işlemler Controller'da olduğu için test zordur. MVP parçalı olduğu için parçalar tek tek birim testle sınanabilir.
- **Görev dağılımı:** Veriyle ilgilenen kısım, görüntüyle ilgilenen kısım ve aradaki parçalar daha iyi ayrılmıştır.
- **Akış:** Bağımlılık döngüsü yoktur. Verinin nereden gelip nereye gittiği rahatça izlenebilir.
- **Presenter'ın rolü:** Görüntüdeki bütün işlemleri kontrol etmez, aradaki veri akışını sağlar. Bir orkestra şefi gibi parçaları uyumlu hale getirir. Görüntü kendi kontrolcüsüne sahip olabilir. Bu, dile ve platforma göre değişir, web ile iOS geliştirme farklıdır.
- **Controller şişmesi:** Massive view controller sorununu çözer.
- **Dezavantaj:** Bileşen sayısı fazladır. Başlangıçta ve basit uygulamalarda uygulaması daha zordur.

10. MVVM (Model-View-ViewModel)

Sunuma göre MVVM daha yenidir, yanlış hatırlanmıyorsa 2005'te ortaya çıkmıştır. Reaktif programlama (reactive programming) üzerine geliştirilmiştir. View ile Model arasına ViewModel sınıfı girer, görev tanımları değişiktir.

View ile ViewModel arasında veri bağlama (data binding) vardır. Bu, bazı dillerde dilin içinde (native) bulunur, bazılarında üçüncü parti kütüphanelerle yapılır. Sunumda Swift için bir kütüphane adı geçti, sonradan native bir karşılığın geldiği söylendi.

> **Ders sonrası not:** Altyazıda "Array Swift" diye geçen kütüphane büyük olasılıkla RxSwift'tir. Sonradan gelen native karşılık da büyük olasılıkla Apple'ın Combine çatısıdır.

Veri bağlamanın mantığı şudur: Bir taraftaki değişikliği diğer taraf izler. Bir taraf değiştiğinde diğerine haber vermesi gerekmez, diğeri zaten izlediği için kendini uyarlar.

Hoca burada bir ekleme yaptı: Association ilişkisini gördüğümüzde kitabın yazarını ve yazarın kitaplarını konuşmuştuk, bu iki alan birbirini referans etsin mi diye sormuştuk. Böyle bir durumda ViewModel yapılabilir. Birden fazla veri tabanı tablosu birleştirilip sanal bir tablo oluşturulabilir. Veri tabanında buna view denir, model tarafında da ViewModel olarak karşılığı olur.

MVVM şu anda özellikle mobil geliştirmede, iOS ve Android tarafında çok yaygındır. Karmaşık uygulamalarda çok işe yarar. Mevcut bir uygulamayı buna geçirmek (refactor) ise çok zordur.

10.1 Görevler ve Veri Akışı

- **Model:** İş mantığını, işlemleri (transaction) ve veri alışverişini üstlenir, verinin modelini oluşturur.
- **ViewModel:** Görüntü mantığını tamamen kendisi oluşturur. Presenter'dan farkı budur: Yalnız aradaki geçişi sağlamaz, görüntü mantığını kurar. View'da bir buton ya da metin kutusu varsa ve onunla iletişim gerekiyorsa ViewModel'da karşılığı bulunmalıdır, çünkü View o değişkeni dinleyecektir.
- **View:** Bildiğimiz görüntüdür. Farkı ViewModel ile veri bağı içinde olmasıdır.

Model, ViewModel'a referans tutmaz. Haber vermesi gerektiğinde uygulama içi bildirim ya da zayıf referans kullanır.

Veri akışı şöyledir:

1. View kullanıcı girdisini alır, ama bir yere iletmez.
2. ViewModel bu değişikliği zaten dinlemektedir. Değişikliğe göre kendi metotlarından gerekenleri çağırır.
3. ViewModel, Model'de gereken değişikliği yapar, sonra kendi içindeki değeri günceller.
4. View, ViewModel'ı izlediği için bu değişikliği anında kendisinde de gösterir.

10.2 Avantaj ve Dezavantajlar

- **Test:** Arayüzü test etmek zordur, otomatik UI testleri uğraştırır. ViewModel test edilerek görüntünün mantığı da test edilmiş olur.
- **Görev dağılımı:** Görevler çok iyi ayrılmıştır.
- **Bağımlılık döngüsü:** Yoktur. Model ile ViewModel arasında geri dönüşte zayıf bağ, View ile ViewModel arasında veri bağlama vardır.
- **View pasiftir:** ViewModel'ın metodunu çağırmaz, kendindeki değişiklikler otomatik uygulanır.
- **Controller şişmesi:** Bu sorunu da çözer.
- **Dezavantaj:** Reaktif programlama yeni başlayanlar için öğrenmesi ve uygulaması daha zor bir olgudur. İş ilanlarında bu yüzden özellikle aranır.

11. Android Üzerinde Giriş Ekranı Örneği

Öğrenci, GitHub'da bulduğu ve aynı giriş (login) ekranını MVC, MVP ve MVVM ile yazan bir Android örneğini gösterdi.

MVC sürümü: Model bir `User` sınıfıdır, kullanıcı adı ve parola tutar, veri üzerinde işlem yapmaz. Giriş başarılı ve başarısız olduğunda yapılacakların hepsini Controller yapar, View'ı da kendisi değiştirir.

MVP sürümü: Arayüzler (interface) çok kullanılır, zayıf referans işi bunlarla çözülür. Giriş metodu bir arayüzde tanımlanır, bir manager sınıfı bu arayüzü uygular. Presenter View'dan adı ve parolayı alır, manager'a `isLoginSuccessful` diye sorar, sonra View'ın `onLoginResult` metodunu çağırır. View da kendini günceller. Öğrenci, Controller'ın ne kadar büyük olduğunu gördükten sonra Presenter'daki kodun ne kadar küçüldüğüne dikkat çekti. Örnekte Presenter'ın bir yerde arayüz yerine doğrudan sınıfı referans almasını da yanlış buldu.

MVVM sürümü: Model yine `User`'dır. View'da bir `Observer` ile ViewModel kendine bağlanır, hangi elemanın hangi elemanla eşleştiği burada tanıtılır. ViewModel giriş olduğunda kendi içindeki `User` değerini değiştirir, View da otomatik olarak değişmiş olur.

> **Ders sonrası not:** Altyazıda "MutableLight" diye geçen sınıf büyük olasılıkla Android'in `MutableLiveData` sınıfıdır. Öğrenci de bunu tahminle söylediğini belirtti.

Sunumda anlatılan MVP akışının Android'e bağlı olmayan sade hali şöyledir. `KonsolView` burada ekranın yerini tutar:

```
class User {
    private final String username;
    private final String password;

    User(String username, String password) {
        this.username = username;
        this.password = password;
    }

    String getUsername() { return username; }
    String getPassword() { return password; }
}
```

```java
interface LoginManager {
    boolean isLoginSuccessful(User user);
}

class BasitLoginManager implements LoginManager {
    @Override
    public boolean isLoginSuccessful(User user) {
        return user.getUsername().equals("demo")
                && user.getPassword().equals("abc");
    }
}

interface LoginView {
    String getName();
    String getPassword();
    void onLoginResult(boolean basarili);
}

class LoginPresenter {
    private final LoginView view;
    private final LoginManager manager;

    LoginPresenter(LoginView view, LoginManager manager) {
        this.view = view;
        this.manager = manager;
    }

    void login() {
        User user = new User(view.getName(), view.getPassword());
        view.onLoginResult(manager.isLoginSuccessful(user));
    }
}

class KonsolView implements LoginView {
    private final String name;
    private final String password;

    KonsolView(String name, String password) {
        this.name = name;
        this.password = password;
    }

    @Override public String getName() { return name; }
    @Override public String getPassword() { return password; }

    @Override
    public void onLoginResult(boolean basarili) {
        String mesaj = basarili ? "giriş başarılı" : "giriş başarısız";
        System.out.println(name + ": " + mesaj);
    }
}

public class MvpOrnek {
    public static void main(String[] args) {
        LoginManager manager = new BasitLoginManager();
        new LoginPresenter(new KonsolView("demo", "abc"), manager).login();
        new LoginPresenter(new KonsolView("demo", "xyz"), manager).login();
    }
}
```

javac ile derlenip çalıştırıldı, çıktı “demo: giriş başarılı” ve “demo: giriş başarısız” oldu. Presenter yalnız `LoginView` ve `LoginManager` arayüzlerini bilir, ekranın ya da manager’ın gerçek sınıfını bilmez. Sunumda sözü edilen, arayüz üzerinden referans alma budur.

MVVM’deki izleme mantığının sade hali de şöyledir. `Gozlenen` sınıfı `MutableLiveData`’nın yerini tutar:

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;

class Gozlenen<T> {
    private final List<Consumer<T>> dinleyiciler = new ArrayList<>();

    void observe(Consumer<T> dinleyici) {
        dinleyiciler.add(dinleyici);
    }

    void setValue(T deger) {
        for (Consumer<T> d : dinleyiciler) {
            d.accept(deger);
        }
    }
}

class LoginViewModel {
    final Gozlenen<String> sonuc = new Gozlenen<>();

    void login(String name, String password) {
        boolean basarili = name.equals("demo") && password.equals("abc");
        sonuc.setValue(basarili ? "giriş başarılı" : "giriş başarısız");
    }
}

public class MvvmOrnek {
    public static void main(String[] args) {
        LoginViewModel viewModel = new LoginViewModel();
        viewModel.sonuc.observe(
                metin -> System.out.println("Görüntü güncellendi: " + metin));
        viewModel.login("demo", "abc");
        viewModel.login("demo", "xyz");
    }
}
```

javac ile derlenip çalıştırıldı, iki çağrıda da “Görüntü güncellendi: …” satırı kendiliğinden yazıldı. `LoginViewModel` görüntüyü hiç tanımaz, yalnız kendi değerini değiştirir. Görüntü bu değeri izlediği için güncellenir.

12. Clean Architecture

Sunumun sonunda öğrenci, yazılım mühendisliğinde “Uncle Bob” diye bilinen kişinin geliştirdiği Clean Architecture’dan söz etti. Her dile ayrı uyarlandığı için tek bir örnek koymadığını söyledi. Katmanlar dıştan içe şöyledir:

1. En dışta UI, yani arayüz.
2. Sonra Controller’lar.
3. Sonra kullanım senaryoları (use case). Örneğin alışverişte bir butona tıklandı ve sipariş (order) verildi. Sipariş birçok yere haber göndermelidir, bunu use case’ler yapar.
4. En altta varlıklar (entities), yani bizim model diye bildiğimiz şey.

Kural, geriye dönük bilginin olmamasıdır. Entity’nin View’dan haberi olmaz, Controller’ın da View’dan haberi olmaz. View Controller’ı tutar ve kendini ona göre değiştirir. Farklı dillerde

farklı yorumlanabilir. Swift'teki karşılığı VIPER'dır. Ayrıntısına girilmedi.

> **Ders sonrası not:** "Uncle Bob" büyük olasılıkla Robert C. Martin'dir.

13. Veri Tabanı Sorguları, ORM ve Yazılım Tasarımı

Hoca sunumdan sonra öğrencilerin başka bir dersinde yapacakları basit projeye bağlayarak dersin toparlamasını yaptı. Veri tabanını tasarlamak önemlidir, ama sorgu çekmek çok zor bir iş değildir, çünkü bunu günümüzde çatılar (framework) yapar. `SELECT * FROM` bilmek gerekir, ama abartılacak bir şey değildir. .NET'te Entity Framework, Java'da Hibernate gibi teknolojiler bu işi üstlenir: "Bütün kitapları getir" dersiniz, hiç `SELECT` yazmadan veri gelir. Çatı `SELECT` cümlesini arka planda kendisi yazar.

> **Hocanın vurgusu:** İşin önemli tarafı veri tabanının iyi organize edilmesi ve nesneye yönelik tasarımın iyi yapılmasıdır. Hangi sınıflar olacak, aralarındaki ilişkiler nasıl olacak, kod bir yerlerde şişecek mi? Kod bu tür kusurlardan (code smell) uzak olmalıdır.

Büyük bir projede veri tabanını veri tabanı tasarımcıları, nesneye yönelik kısmı yazılım tasarımcıları düzenler. Uygulama web ise webe, mobil ise mobile, masaüstü ise masaüstüne uygun bir mantıkla çalışır ve modülerlik böyle sağlanır.

Bu derste nesneye yönelik programlamanın en temelleri konuşuldu. Yazılım mühendisliği tarafına girmek isteyenlerin bilmesi gerekenleri hoca şöyle saydı:

- Veri tabanı.
- Web programlama ve mobil programlama.
- Nesneye yönelik programlamanın bir üst seviyesi olan tasarım kalıpları ve yazılım tasarımı.

Hoca bu konuda Türkçe kaynakların da artık iyi olduğunu söyledi, kitabı olan yazarlar andı.

Mevzu, iyi bir tasarımın nasıl yapılacağına gelir. Dil ne olursa olsun, hoca bunu kısaca UML'i en uygun şekilde tasarlamak diye ifade etti. Bu işte uzmanlaşan birinin veri tabanı tasarımcısı olması gerekmez, onu başkası yapar. Büyük projelerde her şeyin altından bir kişi kalkmaz, modülerlik ve iş paylaşımı vardır.

Dersin Toparlaması

Dersin ilk yarısında üretici-tüketici problemi `LinkedList`, `synchronized`, `wait` ve `notify` ile kuruldu. Derste görülen asıl zorluk senkronizasyonun kendisinden çok, tüketicinin ne zaman duracağına karar vermekti. Tek sayaçla kurulan koşullar ya son elemanları tüketmedi ya da thread'i sonsuza kadar bekletti. Ardından Swing ile pencere, buton ve metin kutusu kuruldu. `ActionListener` ile olay yönetimi üç yolla yazıldı ve çok sayıda buton için `getSource` ile ortak bir metot kullanıldı.

İkinci yarıda bu kodun neden tek sınıfta durmaması gerektiği görüldü. Katmanlı mimari veri, iş mantığı ve arayüzü ayırdı, MVC bunu Model, View ve Controller olarak somutlaştırdı. Öğrenci sunumunda MVP'nin Controller'ı nasıl daralttığı, MVVM'nin veri bağlama ile View'ı nasıl pasifleştirdiği ve Clean Architecture'ın bağımlılık yönünü nasıl sınırladığı anlatıldı. Hoca, bu dersin temelleri verdiğini, bundan sonraki adımın tasarım kalıpları ve iyi yazılım tasarımı olduğunu söyleyerek dersi kapattı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.